

# Tutorial: Desenvolvimento de Aplicação Java Web com JSP e JDBC

---

Este tutorial guia você passo a passo na criação de uma aplicação Java Web para gerenciar uma lista de jogadores de um time, utilizando Maven, Jakarta EE, JSP e banco de dados MySQL.

---

## 1. Estrutura do Projeto e Dependências

O projeto utiliza o **Maven** para gerenciar dependências e o plugin do **Jetty** para facilitar a execução sem a necessidade de instalar um servidor externo manualmente.

Conteúdo do `pom.xml`

Certifique-se de que seu arquivo `pom.xml` contém as dependências fundamentais: `jakarta.servlet-api`, `jakarta.servlet.jsp.jstl` (para suporte a tags JSTL) e `mysql-connector-j` (para conexão com o banco).

---

## 2. Configuração do Banco de Dados

Antes de iniciar o código Java, você deve preparar o banco de dados MySQL. Use o script abaixo no seu terminal MySQL ou Workbench:

```
-- Criar o banco de dados
CREATE DATABASE IF NOT EXISTS time;
USE time;

-- Criar a tabela de jogadores
CREATE TABLE IF NOT EXISTS jogadores (
    idJogador INT AUTO_INCREMENT PRIMARY KEY,
    nomeJogador VARCHAR(100) NOT NULL,
    apelidoJogador VARCHAR(50),
    posicaoJogador VARCHAR(50),
    idadeJogador INT
);

-- Inserir alguns dados de teste
INSERT INTO jogadores (nomeJogador, apelidoJogador, posicaoJogador, idadeJogador)
VALUES
('Neymar Jr', 'Ney', 'Atacante', 32),
('Vinícius Jr', 'Vini', 'Ponta', 23),
('Alisson Becker', 'Alisson', 'Goleiro', 31);
```

---

## 3. Classe de Conexão (`ConnectionFactory.java`)

A classe `ConnectionFactory` é responsável por estabelecer o canal de comunicação entre sua aplicação e o MySQL.

**Local:** `src/main/java/dao/ConnectionFactory.java`

```
package dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionFactory {
    public Connection getConnection() throws ClassNotFoundException {
        final Connection conn;
        // Carrega o driver JDBC do MySQL
        Class.forName("com.mysql.cj.jdbc.Driver");
        try {
            // URL de conexão: jdbc:mysql://servidor/nome_banco
            conn = DriverManager.getConnection("jdbc:mysql://localhost/time",
"root", "1234");
        } catch (SQLException ex) {
            throw new RuntimeException("Erro ao conectar ao banco: " +
ex.getMessage());
        }
        return conn;
    }
}
```

[!NOTE] Ajuste o usuário (`root`) e a senha (`1234`) conforme a configuração do seu MySQL local.

## 4. O Modelo (`Jogador.java`)

Representa o objeto do mundo real (ou da tabela) dentro do seu código.

**Local:** `src/main/java/model/Jogador.java`

```
package model;

public class Jogador {
    private String nome;
    private String apelido;
    private String posicao;
    private int idade;

    // Getters e Setters para permitir o acesso aos atributos
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getApelido() { return apelido; }
    public void setApelido(String apelido) { this.apelido = apelido; }
```

```
public String getPosicao() { return posicao; }
public void setPosicao(String posicao) { this.posicao = posicao; }
public int getIdade() { return idade; }
public void setIdade(int idade) { this.idade = idade; }
}
```

---

## 5. Acesso aos Dados (JogadorDao.java)

O DAO (*Data Access Object*) contém as regras de negócio para salvar e listar os dados do banco.

**Local:** `src/main/java/dao/JogadorDao.java`

```
package dao;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import model.Jogador;

public class JogadorDao {
    private final Connection connection;

    public JogadorDao() throws ClassNotFoundException {
        this.connection = new ConnectionFactory().getConnection();
    }

    // Método para listar todos os jogadores do banco
    public List<Jogador> listarJogadores() throws SQLException {
        List<Jogador> lista = new ArrayList<>();
        String sql = "SELECT * FROM jogadores";

        try (PreparedStatement stmt = connection.prepareStatement(sql);
            ResultSet resultSet = stmt.executeQuery()) {

            while (resultSet.next()) {
                Jogador jogador = new Jogador();
                jogador.setNome(resultSet.getString("nomeJogador"));
                jogador.setApelido(resultSet.getString("apelidoJogador"));
                jogador.setPosicao(resultSet.getString("posicaoJogador"));
                jogador.setIdade(resultSet.getInt("idadeJogador"));
                lista.add(jogador);
            }
        } catch (SQLException ex) {
            throw new RuntimeException(ex);
        }
        return lista;
    }
}
```

```
}  
}
```

---

## 6. Configuração Web (`web.xml`)

Define a página inicial da sua aplicação.

**Local:** `src/main/webapp/WEB-INF/web.xml`

```
<?xml version="1.0" encoding="UTF-8"?>  
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="5.0">  
  <display-name>Aplicacao JSP</display-name>  
  <welcome-file-list>  
    <welcome-file>index.jsp</welcome-file>  
  </welcome-file-list>  
</web-app>
```

---

## 7. Páginas JSP (Interface)

As páginas JSP misturam HTML com código Java para exibir os dados dinamicamente.

Página de Lista (`listajogadores.jsp`)

Esta página instancia o DAO, busca os jogadores e os exibe em uma tabela HTML.

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>  
<%@ page import="java.util.List" %>  
<%@ page import="model.Jogador" %>  
<%@ page import="dao.JogadorDao" %>  
  
<!DOCTYPE html>  
<html lang="pt-br">  
<head>  
  <meta charset="UTF-8">  
  <title>Lista de Jogadores</title>  
</head>  
<body>  
  <h1>Lista de Jogadores</h1>  
  <table border="1">  
    <tr>  
      <th>Nome</th>  
      <th>Apelido</th>  
      <th>Posição</th>  
      <th>Idade</th>  
    </tr>  
  <%
```

```
JogadorDao dao = new JogadorDao();
List<Jogador> jogadores = dao.listarJogadores();
for(Jogador jogador : jogadores){
%>
    <tr>
        <td><%= jogador.getNome() %></td>
        <td><%= jogador.getApelido() %></td>
        <td><%= jogador.getPosicao() %></td>
        <td><%= jogador.getIdade() %></td>
    </tr>
    <% } %>
</table>
<a href="index.jsp">Voltar</a>
</body>
</html>
```

---

## 8. Como Executar

1. Abra o terminal na pasta raiz do projeto (onde está o `pom.xml`).
2. Digite o seguinte comando do Maven:

```
mvn jetty:run
```

3. Abra o seu navegador e acesse: <http://localhost:8080>

Parabéns! Você desenvolveu uma aplicação Java Web completa integrando banco de dados com JSP.